Availability and Services

Chapter 11 — Software Architecture (Pautasso)

- **What:** runtime **uptime** and correct behavior under faults — especially when using **services** operated by others.
- **Why:** Lecture **10** added capacity; Lecture **11** asks what happens when parts **fail** or disappear.

> **Definitions** · **Availability** — fraction of time the system correctly serves users. · **Service** — capability you consume over a network, usually operated by another organization.

Learning objectives

- **Components vs services** — *What:* ownership, packaging, remote connectors. *Why:* availability **responsibility** moves with who runs the binary.
- **Measure availability** — *What:* SLIs, probes, incidents. *Why:* “feels fine” is not an **SLO**.
- **Stop cascades** — *What:* timeouts, circuit breaker, canary. *Why:* **good intentions** (retries) can **amplify** outages.
- **Replication trade-offs** — *What:* sync/async replicas; CAP intuition. *Why:* extra copies help **faults** but complicate **consistency**.
- **Event sourcing (intro)** — *What:* append-only facts + replay. *Why:* auditability and recovery patterns for **critical** workflows.

> **Definitions** · **SLI** — service level *indicator*: a measurable signal (e.g. success ratio). · **SLO** — service level *objective*: target over time for that SLI. · **Circuit breaker** — stops calls to a failing dependency for a cooldown. · **Canary (call)** — routes a risky request to one instance first to limit blast radius. · **CAP** — trade-off among consistency, availability, and tolerance of network partitions.

Recap — Lecture 10

- **What:** scalability — workload vs resources; saturation; shards, caches, queues.
- **Why:** a system can be **fast** yet **fragile** — availability is a **different** design axis.

> **Definitions** · **Scalability** — handling growth in workload by adding resources while meeting latency/throughput targets. · **Saturation** — point where adding load no longer increases useful throughput.

Chapter roadmap (from contents)

- **Components vs services** — map component size → SaaS model
- **Monitoring availability** — active vs passive; heartbeats; probe depth
- **Impact of downtime** — money, trust, retries
- **Circuit breaker & canary call** — contain cascades; protect workers
- **Replication** — consistency vs availability; sync vs async
- **Event sourcing** — log facts for replay/audit

> **Definitions** · **SaaS** — software as a service: provider runs the app; you pay for access/use. · **Heartbeat** — periodic “I’m alive” signal from a component to a monitor. · **Replication** — maintaining redundant copies of data or compute for fault tolerance.

Components vs services — the map question

- **What:** a `getMap(lat,lng,zoom)` "component" could imply **planet-scale** map data if shipped **self-contained**.
- **Why:** explains why **offline** delivery (CDs, app bundles) hits **physics** — motivates **remote** map tiles today.

> **Definitions** · **Component** — deployable unit with a clear interface, typically run inside *your* environment. · **Self-contained** — package includes all bits and data needed to run without reaching a remote server.

Service = software someone else runs

- **What:** client sends messages; data and upgrades live **in the provider's** cloud.
- **Why:** you stop shipping terabytes to users — **freshness** and **partial** delivery (city vs world) become feasible.

> **Definitions** · **Remote service** — logic and data hosted by a third party; you integrate via APIs over the network. · **Client/server (SaaS)** — thin client talks to a shared backend maintained by the provider.

Who owns availability?

- **Component (you run)** — *What:* you operate lifecycle; internal **help desk** same org. *Why:* outage is **your** queue.
- **Service (they run)** — *What:* provider controls deploys, pricing, **existence** of API. *Why:* your app's uptime **depends** on theirs — contractual + architectural coupling.

> **Definitions** · **Operational ownership** — who deploys, patches, scales, and carries the pager. · **Vendor coupling** — your correctness and uptime depend on an external party's behavior and SLAs.

Business models (lecture)

- **Per deploy / install** — *What:* pay on download or license seat. *Why:* classic **shrink-wrap** and app-store economics.
- **Per use / subscription** — *What:* pay per call or monthly SaaS. *Why:* aligns revenue with **continuous** operation — changes how you measure **availability**.

> **Definitions** · **Per-deploy pricing** — revenue when a customer installs or upgrades a copy. · **Subscription / pay-per-use** — recurring or metered payment tied to ongoing access or API calls.

Technical: packaging vs publishing

- **Components** — *What:* install into **your** stack; must match **language/framework** (Eclipse vs npm vs apt). *Why:* **compatibility** explosion — Docker standardizes **packaging**, not trust.
- **Services** — *What:* published URL; **HTTP/JSON** lingua franca; remote connector (often async bus). *Why:* integration speed — but dependency is **outside** your firewall.

> **Definitions** · **Connector** — mechanism linking components (RPC, HTTP, message bus). · **COTS** — commercial off-the-shelf: buy vs build for a component. · **Docker (here)** — standard *packaging* format; does not remove trust issues with remote vendors.

Remote connector constraint

- **What:** cannot use **shared memory** with a cloud service — need message/RPC/HTTP.
- **Why:** async patterns often improve **availability** (temporal decoupling) — lecture link to **ESB / message bus** era.

> **Definitions** · **Shared memory** — IPC within one machine; impossible across the public Internet. · **ESB / message bus** — middleware that routes asynchronous messages between services to loosen temporal coupling.

Availability — operational view

- **What:** fraction of time system **delivers correct service** to users (not only “ping OK”).
- **Why:** distinguishes **reachable** from **useful** — aligns with SLIs and user journeys.

> **Definitions** · **Reachable** — host answers TCP/HTTP (process up). · **Useful** — user-visible work succeeds (correct business outcome within SLO).

MTBF and MTTR (intuition)

- **MTBF** — *What:* mean time **between** failures — reliability of components.
- **MTTR** — *What:* mean time **to restore** — ops skill, automation, failover.
- **Why:** availability improves if you **rarely** break **or** you **recover fast** — architecture affects **both**.

> **Definitions** · **MTBF** — mean time between failures: average uptime between incidents (informal use in lectures). · **MTTR** — mean time to repair/restore: downtime until service is back within SLO.

Dependency chain availability

- **What:** your SLO is **multiplied** by dependencies — one weak link caps effective uptime.
- **Why:** CityBite checkout needs **API + DB + payment + SMS** — model the **chain**, not only one tier.

> **Definitions** · **Dependency chain** — sequence of systems all required for one user journey. · **Effective availability** — combined probability all parts in the chain succeed (roughly multiplicative for series dependencies).

Redundancy

- **What:** extra instances, replicas, zones — **N+1** capacity.
- **Why:** survives single failure — must pair with **health checks** and **traffic shift** (failover) or you only own spare **cost**.

> **Definitions** · **Redundancy** — duplicate components or data so one failure does not stop the whole system. · **Failover** — automatic or manual shift of traffic/workload to healthy standby units. · **N+1** — spare capacity beyond minimum *N* needed for normal load.

Monitoring — why observe?

- **What:** active probes or passive heartbeats to infer **up/down/degraded**.
- **Why:** do not wait for **Twitter** to learn you are down — detect before users abandon carts.

> **Definitions** · **Monitoring** — collecting signals and alerts about system health. · **Degraded** — partially functional (slower, subset of features) vs fully up or hard down.

Active monitor (watchdog probes)

- **What:** synthetic requests from outside; timeout → alert.
- **Why:** sees network + DNS + TLS + app path — closer to **user** perspective.

> **Definitions** · **Active probe / synthetic check** — monitor initiates a test request from outside the service. · **Watchdog** — observer that expects timely responses or raises an alarm.

Passive / heartbeat

- **What:** service periodically **pings** monitor; silence implies failure.
- **Why:** cheap **liveness** signal — still need **deep** checks for “can serve traffic” (pool exhaustion).

> **Definitions** · **Heartbeat** — periodic message proving a process is still running. · **Liveness** — “is the process alive?” (restart if not). · **Readiness** — “can this instance accept traffic now?” (e.g. DB reachable, migrations done).

Watchdog cannot watch itself

- **What:** need redundancy in monitoring; clock skew and **probe cost** matter.
- **Why:** blind spots become incidents — **SRE** meta-availability problem.

> **Definitions** · **Blind spot** — failure the monitoring system cannot see (including monitor down). · **SRE** — site reliability engineering: practices for reliability, SLOs, and incident response.

Downtime impact

- **What:** lost revenue per second; SLA credits; brand; regulatory reporting for payments/health.
- **Why:** justifies **error budgets** and **incident** investment — not “soft” quality.

> **Definitions** · **Downtime** — period when service is outside SLO or unavailable to users. · **SLA** — service level *agreement*: contract with customer/vendor remedies (credits). · **Error budget** — allowed unreliability within an SLO window; spending it triggers process changes.

Overload is an availability incident

- **What:** saturation (L10) causes errors — users see **unavailable** even if “no deploy”.
- **Why:** autoscaling and **queues** are availability tools, not only performance tools.

> **Definitions** · **Overload** — demand exceeds capacity; latency spikes and errors rise. · **Backpressure** — mechanism to slow producers when consumers cannot keep up.

Cascading failure

- **What:** failure spreads because **retries**, thread blocking, and shared pools multiply load.
- **Why:** the **fix** (retry harder) becomes the **attack** — must engineer **backpressure**.

> **Definitions** · **Cascading failure** — one fault triggers failures in other parts (often amplified by retries and resource exhaustion). · **Retry storm** — many clients retry at once and overload a already-failing dependency.

Circuit breaker

- **What:** connector state machine: **closed** → failures trip **open** → fail fast; later **half-open** trial.
- **Why: stops** hammering sick dependencies — saves threads; see `example1_availability_circuit_breaker_citybite.py`.

> **Definitions** · **Circuit breaker (states)** — **Closed**: calls pass. **Open**: fail fast, no calls. **Half-open**: trial call to test recovery. · **Fail-fast** — return error immediately instead of waiting on a doomed dependency.

Canary call

- **What:** send **suspicious** work to **one** worker first; only fan out if safe.
- **Why:** protects fleet from **poison** messages — coal-mine metaphor; trades latency vs **blast radius**.

> **Definitions** · **Canary call** — send a potentially dangerous or expensive request to a single worker first; proceed only if safe. · **Poison message** — input that crashes or hangs workers if processed naïvely. · **Blast radius** — how much of the system one fault can take down.

Replication — why copies?

- **What:** multiple data locations for **read scale** and **fault** tolerance.
- **Why:** disk/host loss should not mean **permanent** data loss — if protocol is sound.

> **Definitions** · **Replica** — copy of data or service state kept in sync (loosely or strongly) with the primary. · **Fault tolerance** — system continues operating (possibly degraded) when components fail.

Sync vs async replication

- **Sync** — *What:* write acknowledged only if **all** required replicas durable.
- **Why: strong** consistency; higher write latency — CAP: favors **consistency** under partition story.
- **Async** — *What:* acknowledge after primary; replicas **lag**.
- **Why:** faster writes; readers may see **stale** data — explicit product decision.

> **Definitions** · **Synchronous replication** — primary waits for replicas before acknowledging the write. · **Asynchronous replication** — primary acknowledges before replicas catch up; **replication lag** is normal. · **Stale read** — read sees an older value than the latest committed write on the primary.

CAP (high level)

- **What:** under network **partition**, choose trade-offs among **C**onsistency, **A**vailability, **P**artition tolerance.
- **Why:** explains why **banking** paths differ from **social like** counters — set expectations with PM.

> **Definitions** · **Consistency (CAP sense)** — every read receives the most recent write or an error. · **Availability (CAP sense)** — every request receives a non-error response (not guaranteeing freshest data). · **Partition** — network drops or delays messages between nodes; system must still choose behavior.

Event sourcing (intro)

- **What:** store **events** append-only; **state** is a projection / fold over history.
- **Why:** replay after bugs; audit “who changed order status when gateway flapped” — complements **availability** narratives (recovery).

> **Definitions** · **Event sourcing** — persist business changes as an ordered append-only *event log* instead of only overwriting current state. · **Projection / read model** — materialized view built by processing events for fast queries.

Example 1 — circuit breaker (`example1_*.py`)

- **What:** flaky **payment gateway** vs naive retries vs breaker **fail-fast** windows.
- **Why:** concrete numbers for slides — “retries are load generators.”

> **Definitions** · **Naive retry** — repeat failed remote call immediately up to N times (can amplify load). · **Bulkhead** — isolate resource pools so one dependency cannot exhaust all threads/connections.

Example 2 — probes (`example2_*.py`)

- **What:** shallow health vs **pool-aware** readiness.
- **Why:** ties lecture to **Kubernetes** readiness you already teach in L9.

> **Definitions** · **Shallow health** — endpoint returns OK if process is up, without checking critical dependencies. · **Deep readiness** — probe validates dependencies needed to serve real traffic (e.g. DB pool slot).

Assignment — CityBite “Always Open”

- **What: 150 pts** — services inventory, SLO/SLI, monitoring, cascades/replication, diagram.
- **Why:** practices **availability language** with the same product story — see `ASSIGNMENT.md`.

> **Definitions** · **Deliverable** — artifact you submit (markdown, diagram) proving analysis. · **Rubric** — grading criteria in `ASSIGNMENT.md` describing required depth.

References (chapter / course)

- Pautasso — **Chapter 11** PDF (`11_Availability_and_Services.pdf`)
- Optional depth: **Release It!** (Nygard), **Site Reliability Engineering** (Google), **DDIA** (Kleppmann)

> **Definitions** · **Reference architecture / book** — authoritative source for patterns and vocabulary used in the course.

Takeaways

- **Services move blame lines** — contract and monitor **vendors** like internal tiers.
- **Availability = measure + contain + recover** — not only “more servers.”
- **Retries + no timeouts = weapon** — pair with **breakers** and **meaningful** probes.

> **Definitions** · **Contain** — limit how far a failure spreads (breakers, timeouts, bulkheads). · **Recover** — restore service within MTTR targets (failover, rollback, replay).

Questions?

- **What:** Chapter **11** + `lect11.txt` (strip NULs if needed).
- **Why:** Next — **Lecture 12** — **flexibility** and **microservices** jump from services.

> **Definitions** · **Microservices** — fine-grained services aligned with bounded contexts (preview of next chapter). · **Flexibility** — ease of evolving the architecture and organization around services.